



Course Code : AIT202

Course Name : Methods and Applications of Deep Learning

Lecturer : Raheel Zafar

Academic Session : 202504

Assessment Title : Deep Learning for Practical Problems: A Group Project

Submission Due Date : 21/06/2025

Prepared by :	Student ID	Student Name
	DSC2302025	Sean Wong Wee Kai
	DSC2304093	Bong Xuan Jing
	DSC2302069	Lim Zee Ern
	DSC2304085	Woon Kai Fan
	DSC2302196	Orynkul Alikhan
	DSC2304098	Onn Heng Hwa

Date Received : 21/06/2025

Feedback from Lecturer:	Mark:
-------------------------	-------

Own Work Declaration

I/We acknowledge that my/our work will be examined for plagiarism or any other form of academic misconduct, and that the digital copy of the work may be retained for future comparisons.

I/We confirm that all sources cited in this work have been correctly acknowledged and that I/we fully understand the serious consequences of any intentional or unintentional academic misconduct.

In addition, I/we affirm that this submission does not contain any materials generated by AI tools, including direct copying and pasting of text or paraphrasing. This work is my/our original creation, and it has not been based on any work of other students (past or present), nor has it been previously submitted to any other course or institution.

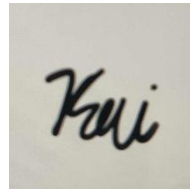
Signature:



(Sean Wong Wee Kai)



(Bong Xuan Jing)



(Woon Kai Fan)



(Lim Zee Ern)



(Orynkul Alikhan)



(Onn Heng Hwa)

Date: 21/6/2025

1.0 Introduction

Facial expression recognition (FER) is a specialized area of computer vision that focuses on analyzing human emotions from facial images or videos. By detecting subtle changes in facial features, such as eyebrow movements, lip curvature, and eye widening, FER systems can classify emotions like happiness, anger, sadness and surprise. This technology has many and wide applications, from improving human-computer interaction and mental health diagnostics to enhancing customer experience analysis and security systems.

Deep learning has transformed FER by automating feature extraction and improving accuracy. Convolutional Neural Networks (CNNs) excel at learning hierarchical patterns from low-level edges to high-level emotional cues directly from raw pixel data. Modern approaches leverage architectures like ResNet or EfficientNet, often pretrained on large datasets to handle challenges such as lighting variations, occlusions (e.g. glasses or beards), and cultural differences in expression.

This project specifically addresses the comparative analysis of Convolutional Neural Networks (CNNs) and ResNet architectures for image detection tasks. Our primary objective is to evaluate the performance and suitability of both a custom-built CNN and a pre-trained ResNet model for facial expression recognition. The significance of this work stems from understanding the strengths and weaknesses of different deep learning architectures in real-world image detection applications, which has broad utility in industries like healthcare, autonomous driving, and security, and can automate critical visual inspection processes.

2.0 Literature Review

2.1 Theoretical Background

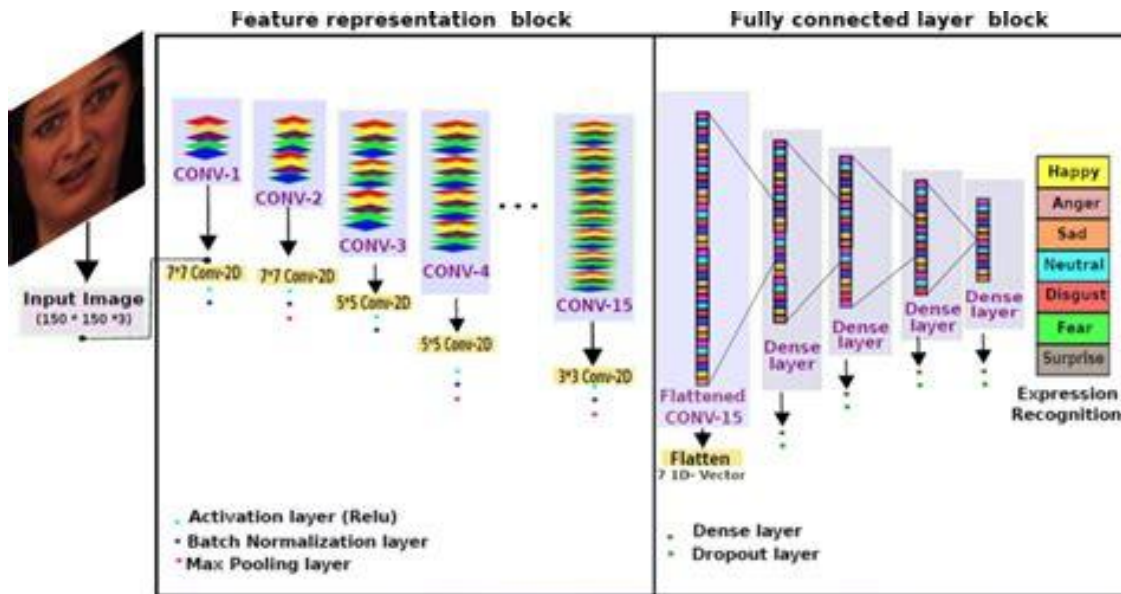
Facial emotion recognition (FER) is a key part of affective computing and is still very important in many areas, including security systems, educational technology, clinical diagnostics, and human-computer interaction. Recently, deep learning methods have made emotion classification systems much more accurate and useful. This is especially true for Convolutional Neural Networks (CNNs) and Residual Networks (ResNets). This review thoroughly evaluates both architectural approaches, as implemented in a cited codebase employing a customized CNN and a carefully optimized ResNet-18 model.

This analysis is organized into six parts: (1) theoretical framework, (2) network architecture, (3) empirical evaluation, (4) practical implementations, (5) comparative insights, and (6) developing hybrid innovations. That way, it can allow for a balanced examination of the models' accuracy and generalization capabilities, as well as their architectural justifications, deployment feasibilities, and alignment with contemporary research advancements.

2.2 CNN Architectures in Emotion Classification

Proposed CNN Architecture for FER

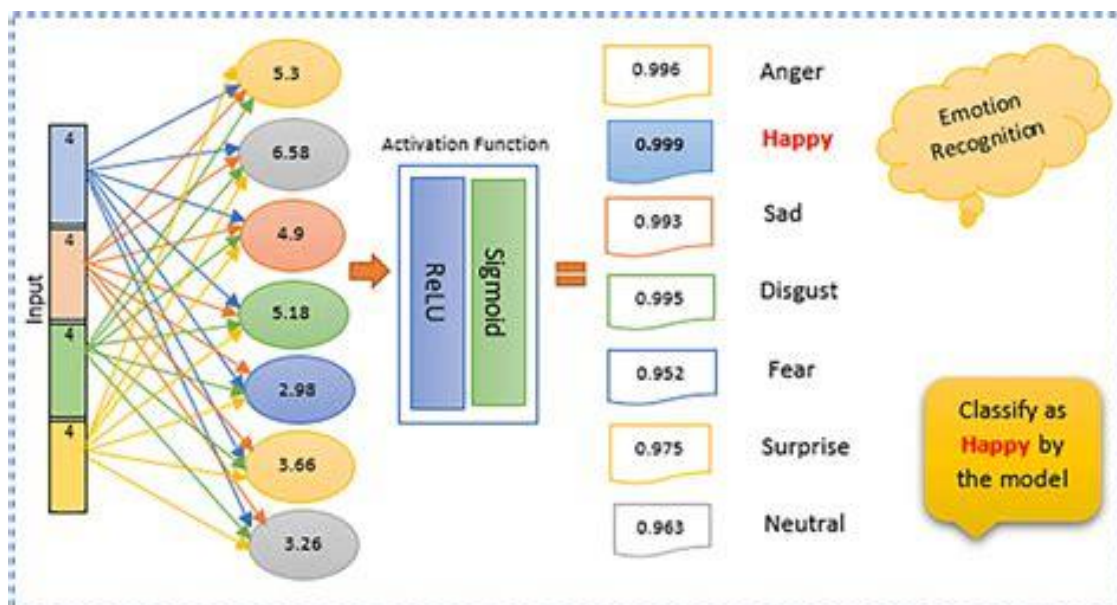
A standard convolutional neural network architecture utilized for facial emotion recognition is shown in the figure below. It is designed with fully connected layers, convolutional layers and activation functions to categorize emotions effectively.



Proposed CNN Architecture

Activation and Output Mapping in CNN

In the image below, the application of ReLU in intermediate layers and sigmoid functions in output layers is shown. It can facilitate multi-class emotional prediction.



Activation and Output Mapping

For tasks that require image recognition, Convolutional Neural Networks (CNNs) are still vital since they are simple to use and process quickly. For recognizing emotions on

faces, they are great at picking up on important visual clues, such as moving eyebrows or smiling, that show how someone is feeling.

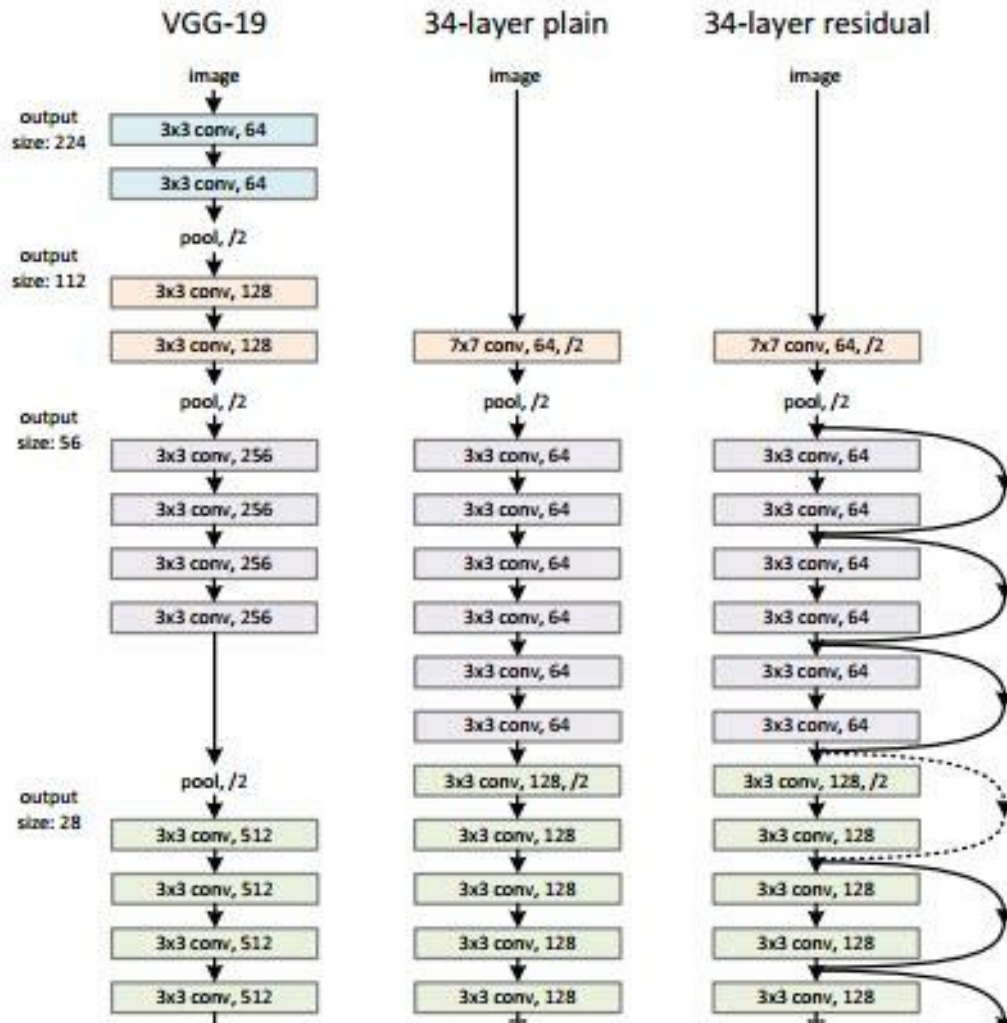
Based on recent advancements in CNN models, it was shown that even minimalistic designs can be rather effective. Gursesli et al. (2024) utilizing depth wise separable convolutions had produced good results which reduces the computing cost. On devices with limited computing resources, Deramgozin et al. (2023) managed to make things work better by utilizing FACS and attention processes.

To improve model reliability, noise injection techniques were utilized by Reddy et al. (2023) which helped CNNs work better with various datasets. Preprocessing Methods like histogram equalization, landmark alignment, and geometric tweaks have made face characteristics clearer hence enabling CNNs to easily detect subtle expressions.

CNNs differ from other types of FER since their later layers detect more abstract emotional qualities while their early layers detect edges and textures. The CNNs' framework helps to spot the difference between expressions that look alike such as fear and surprise or rage and contempt which is also another reason why they perform well in real-world situations.

Residual Network Module (Plain vs. Skip-Connected)

In this diagram, comparisons of training stability and performance of 34-layer residual networks, 34-layer plain networks, and VGG-19 are shown. For deep FER models, it is important to skip connections in residual architectures in order to solve vanishing gradient problems and improve feature propagation.



Residual Network Module

2.3 Residual Networks and Transfer Learning

Real-Time FER CNN Pipeline

Preprocessing and feedforward stages in a real-time FER setup including image normalization, cropping, and CNN-based emotion inference is shown in this pipeline.

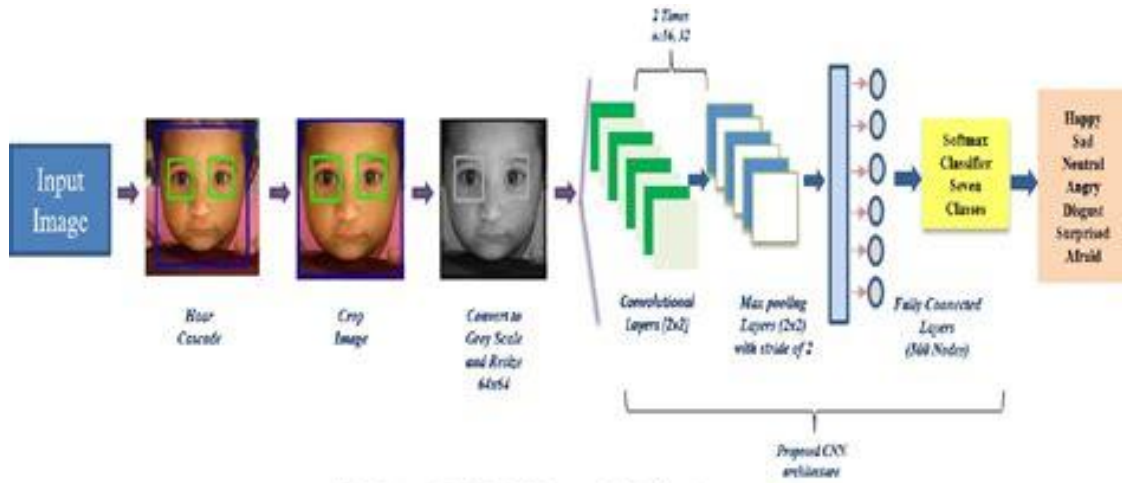


Fig. 3. Proposed CNN model diagram for facial emotion recognition

Real-Time FER Pipeline

In benchmark tasks, Residual Networks (ResNets) when pre-trained on large datasets and then adjusted for particular goals is superior to conventional CNN architectures. ResNet-18, a small model from the ResNet family despite of its small size still has the fundamental advantage of residual learning.

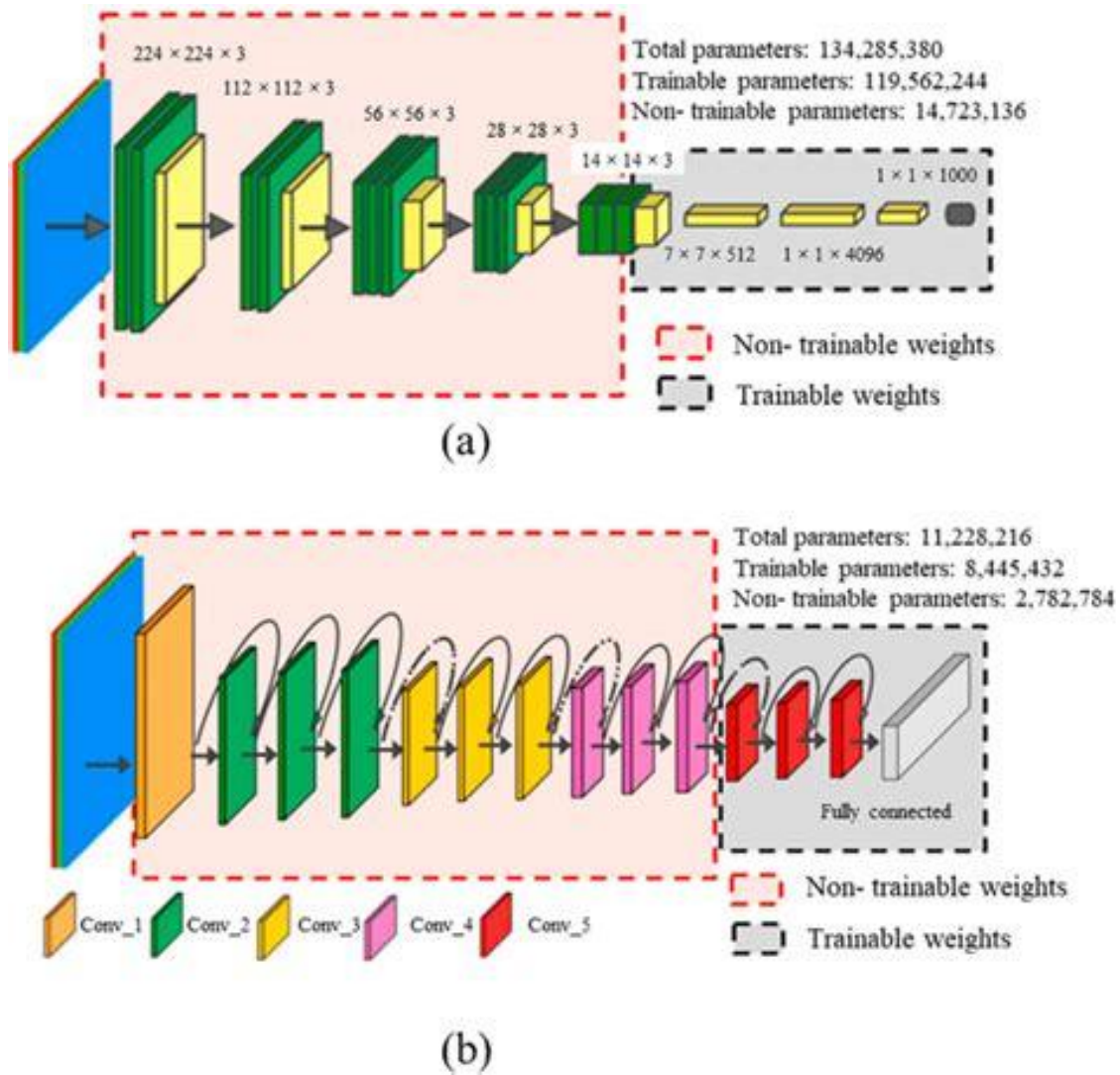
To degradation problem in deep neural networks, shortcut connections are created in residual learning. Instead of completing transformations, improving convergence and classification accuracy, these connections enable the network to learn modifications to identity mappings in complex tasks like facial emotion recognition.

Transfer learning is widely used in deep learning. By optimizing ResNet models that have already been trained on ImageNet using FER-specific datasets, the model's ability to recognize high-level, abstract features relevant to emotion classification can be leveraged by the researchers. During fine-tuning, the later convolutional layers are modified for task-specific learning, while the initial layers—which frequently capture broad patterns—are usually frozen.

According to Ibrahim et al. (2024), deeper architectures, such as ResNet-50, require more computing power but produce higher accuracy than shallower models. As a result, efficiency and model performance are traded off. More sophisticated transfer learning techniques now use domain adaptation to overcome domain differences between source and target datasets. By aligning feature distributions, methods like feature normalization and adversarial training have been successful in enhancing model robustness across a variety of data sources.

Transfer Learning Architecture (VGG-16 & ResNet-18)

The analysis of transfer learning setups in Section 3 is reinforced by this high-resolution schematic, which shows how higher layers are fine-tuned (trainable, in dashed boxes) while early convolutional layers are frozen (non-trainable, in red blocks).



Transfer Learning Architecture

2.4 Performance Benchmarks and Comparisons

Meaningful trade-offs that affect model selection across various use cases are revealed by comparative analyses of FER architectures. ResNet models outperformed

MobileNetV2 and VGG-16 in benchmarking tests by Bi et al. (2025), especially for less distinct emotions like sadness and disgust, in terms of precision and F1 scores. According to Qian et al. (2024), DenseNet and GoogLeNet are consistently being outperformed by ResNet-50 confirming its reputation as a dependable option for emotion classification.

These practical insights can be benefited from by deployment strategies. In applications where speed and energy efficiency are critical like embedded systems or mobile platforms, lightweight CNNs are generally preferred. On the other hand, ResNet models have their own advantages too due to their depth and feature extraction capabilities in situations that demand high accuracy and robustness, like psychological evaluation tools or surveillance systems.

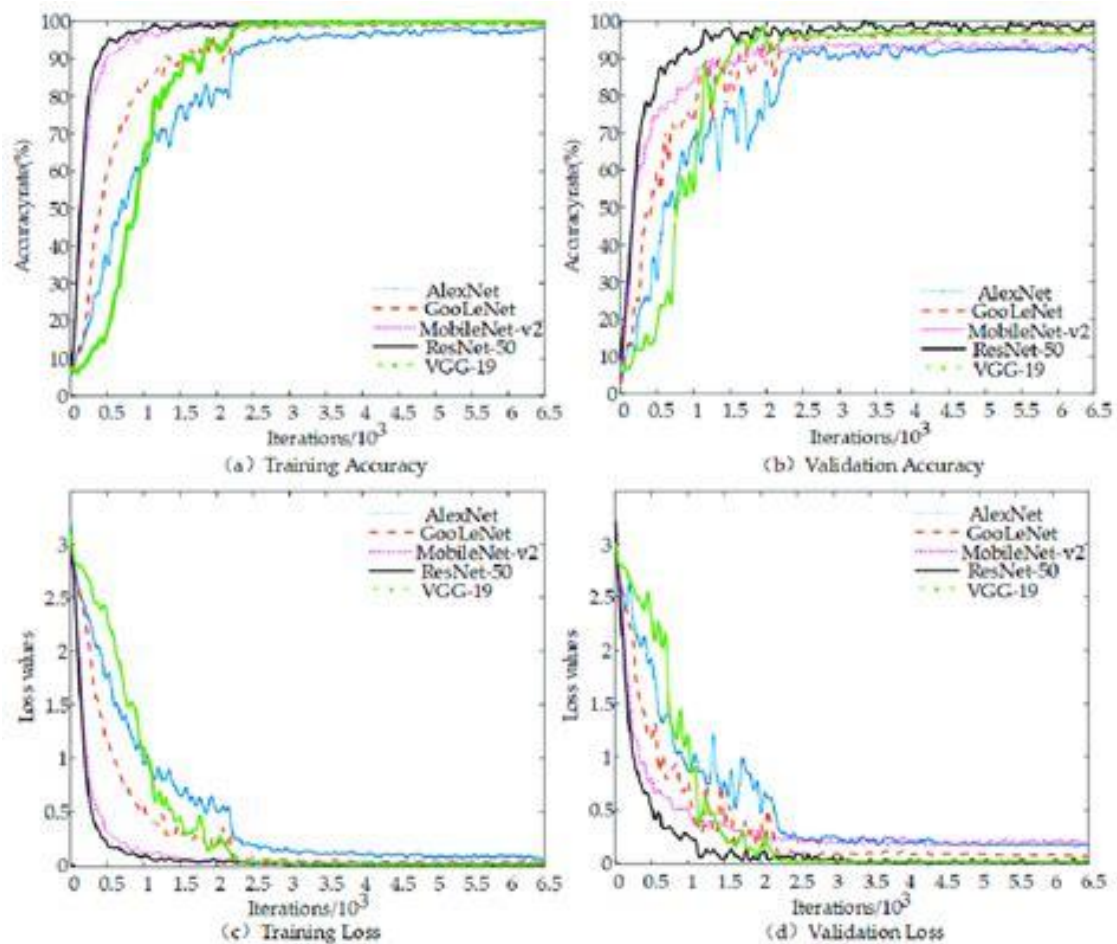
Additional research was conducted by Dhoot et al. (2023) in which 2D and 3D CNNs were contrasted, and they had found that temporal dynamics are better captured by 3D CNNs, whereas increased computational efficiency and better suitability for real-time inference are attributed to 2D CNNs. Hence, the potential advantages of hybrid models are demonstrated, which can lead to improved performance in video-based FER tasks by combining efficiency and temporal sensitivity.

The best balance between processing speed and accuracy in a real-time student engagement detection system was offered by ResNet-50, according to applied research by Gupta et al. (2022).

Visual Performance Comparisons

Comparison of ResNet-50 with Other CNN Models

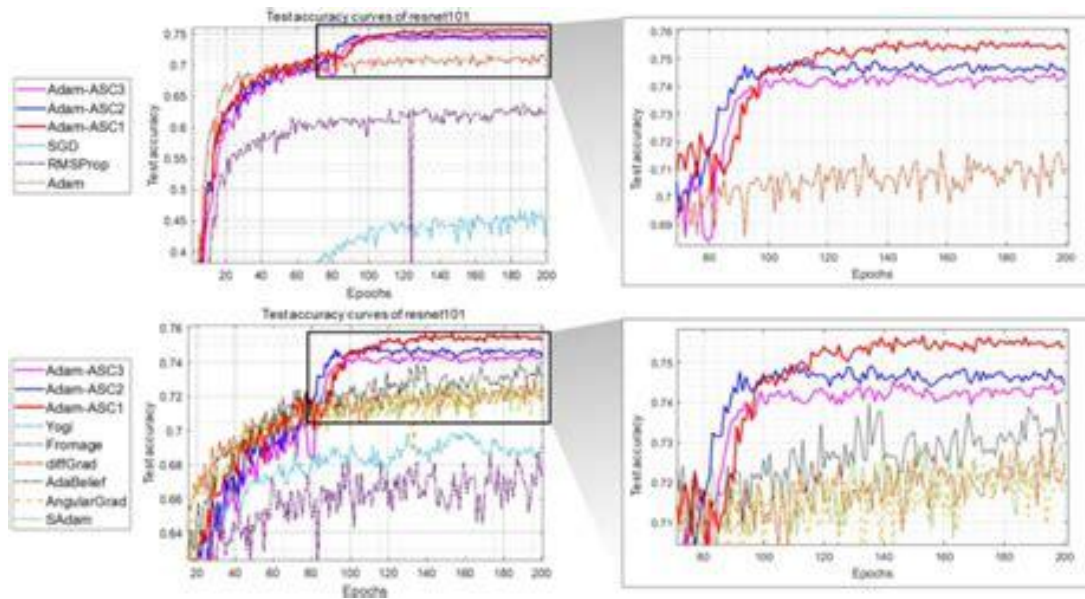
This chart highlights how ResNet-50 compares in training progression and classification strength against alternatives like VGG16 and Inception.



Comparison of CNN Models

ResNet-101 vs. ResNet-50 Accuracy Curves.

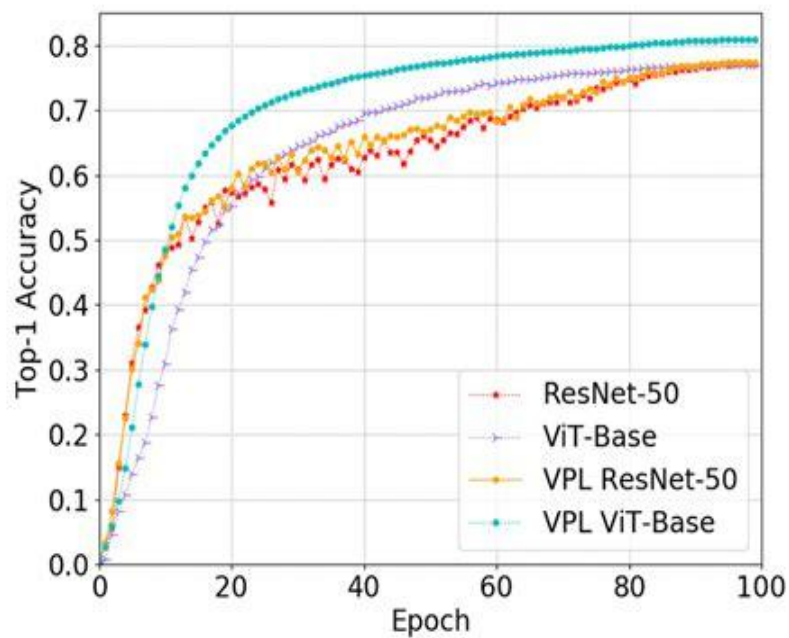
An instructive illustration of how depth (ResNet-101) affects convergence and slightly increases accuracy when trained using optimal techniques



ResNet-101 vs. ResNet-50

Top-1 Accuracy Comparison: ResNet-50 vs. Vision Transformers

ResNet-50 shows competitive convergence to around 80% Top-1 accuracy within 100 epochs, providing a valuable performance baseline against modern architectures like ViT.



ResNet vs ViT Accuracy

The trade-offs in model selection are highlighted by comparative studies. Bi et al. (2025) benchmarked several models and discovered that ResNet outperformed MobileNetV2 and VGG-16 in terms of precision and F1 scores, especially when it came to more ambiguous emotions like disgust and sadness. Qian et al. (2024) confirmed the dependability of ResNet50 as it demonstrated better performance than DenseNet and GoogLeNet.

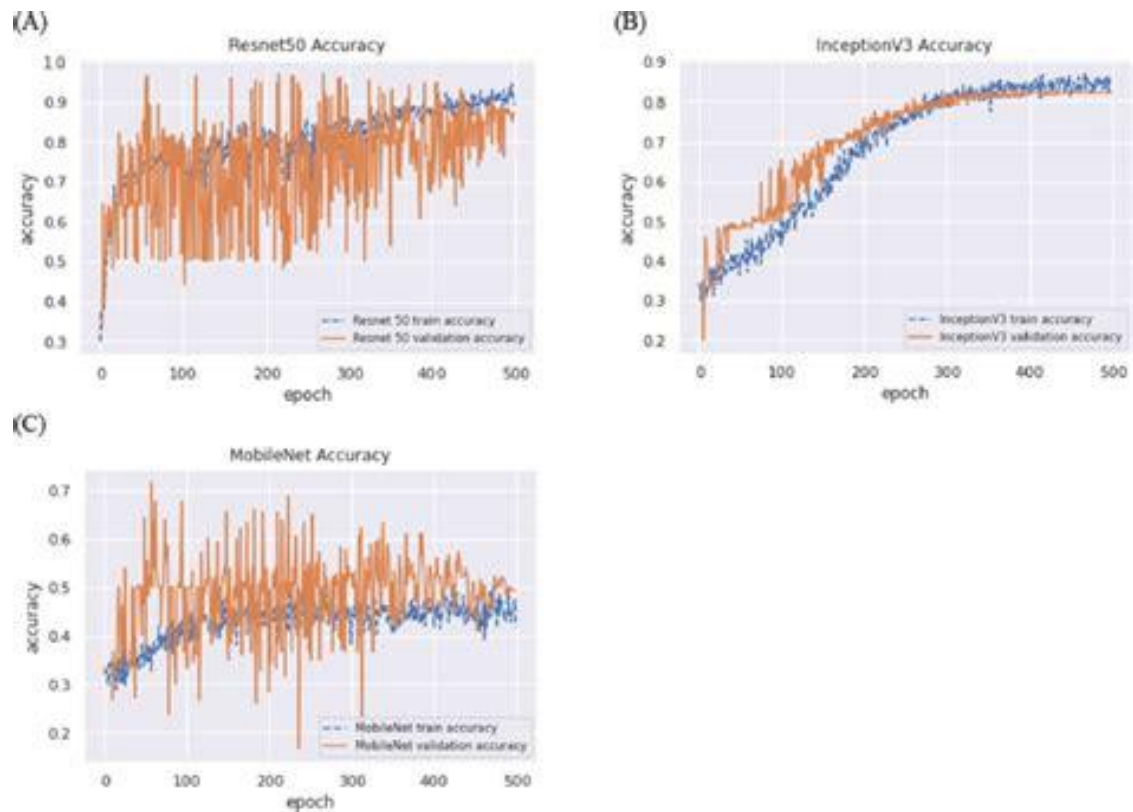
From these results, it can provide an idea on how the choice of architecture in real-world implementations is decided. For instance, in situations where low power consumption and real-time inference are prioritized, lightweight CNNs are preferred. Alternatively, in situations where precision and resilience is emphasized, such as in security systems or psychological evaluations, ResNet models are a better choice.

From their comparison of 2D and 3D CNNs, it was observed by Dhoot et al. (2023) that 2D CNNs are better suited for real-time applications because of their lower computational overhead. Nonetheless, 3D CNNs still has superior temporal feature extraction, indicating their potential for use in video-based FER. By combining these advantages, hybrid models could produce systems that are effective and able to analyze time.

It was shown by Gupta et al. (2022) that the ideal balance between speed and accuracy in real-time student engagement detection was struck by ResNet-50. It is emphasized how critical it is for model complexity to be matched with application requirements, a principle that is considered particularly pertinent given the growing integration of FER technology into commonplace goods and services.

Training and Validation Accuracy Trends

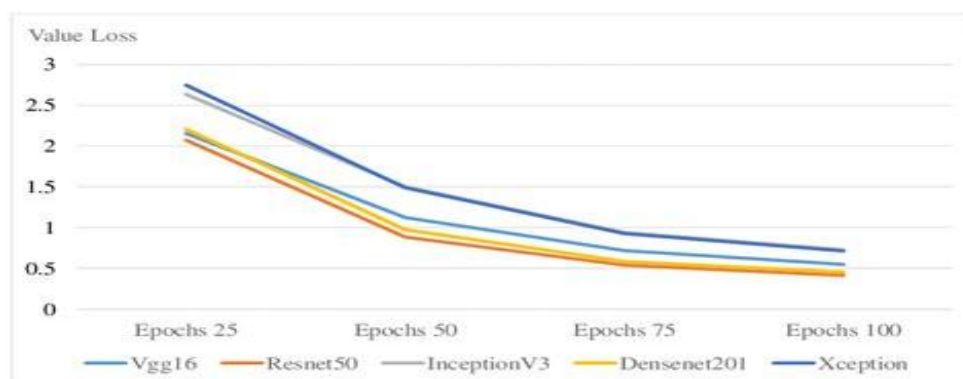
Convergence behavior for ResNet-50 across training and validation sets is shown on the images below emphasizing strong generalization.



Training vs. Validation Accuracy

Model Loss Across Architectures

Epoch-wise loss graphs for VGG-16, ResNet-50, InceptionV3, DenseNet-201, and Xception. The balance of accuracy and efficiency is aligned with the faster and smoother decline of ResNet-50.



Loss Comparison Chart

2.5 Hybrid and Attention Mechanisms

Complementary features from several model types are combined by hybrid architectures, which are considered a promising advancement in facial emotion recognition (FER). To capture spatial features and temporal dynamics at the same time, a concept was proposed by Mehta and Bhalla (2024) that combines Convolutional Neural Networks (CNNs) with Recurrent Neural Networks (RNNs), specifically Long Short-Term Memory (LSTM) units. This allows for the interpretation of how emotions change over time by using CNNs to extract frame-wise facial features and LSTMs to process the sequence.

This hybrid approach can be taken a step further by incorporating attention mechanisms. The mechanism focuses on facial features that are particularly expressive, such as the mouth, eyes, or eyebrows. Model interpretability and predictive accuracy are enhanced by attention layers through the assignment of varying weights to different facial features according to their significance to the emotional state.

Applications where emotional cues are fleeting or subtle, like interactive robotics or mental health assessment tools are well-suited for this architectural fusion. Since attention mechanisms had already shown their usefulness in more general computer vision domains, the incorporation of these mechanisms into transformer-based systems may be proven advantageous for future research. The limits of what FER systems can accomplish in emotionally complex and real-time environments could be improved by these models through a more scalable and data-efficient combination of spatial and temporal modelling.

2.6 Dataset and Evaluation Metrics

A number of publicly accessible datasets are frequently used to assess facial emotion recognition (FER) systems; each dataset has unique features that affect model training and evaluation. For example, FER-2013 includes photos taken in real-world settings, which introduces variations in lighting, angles, and occlusion. CK+ is appropriate for researching distinct emotions because it includes high-resolution facial expressions captured in lab-controlled settings. AffectNet distinguishes itself by providing a wide

range of richly annotated emotion categories, including subtle and compound emotions, whereas JAFFE focusses on expressions from Japanese female subjects.

Accuracy, precision, recall, F1-score, and confusion matrices are common evaluation metrics for FER. These metrics are good for giving a thorough idea of how well the model performs.

Class imbalance is a common problem in which certain emotions like "happy" or "neutral" are over-represented while others like "disgust" or "fear" are under-represented. This will cause biased learning if not handled properly. To deal with this problem, mitigation techniques like weighted loss functions, augmentation methods, and synthetic data generation are frequently used.

The credibility of the model can also be verified by using cross-dataset validation. An example is a model trained on FER-2013 is tested using data from AffectNet or CK+. Accuracy and dependability are more likely to be retained in real-world scenarios by models that perform well across datasets. This is because unpredictable and varied input conditions are frequently involved in real-world deployment.

2.7 Practical Applications and Deployment

In practical contexts like education, car safety, and mental health monitoring, the importance of facial emotion recognition (FER) should not be underestimated.

Constraints must be taken into consideration when FER is deployed in these settings. Important considerations include inference time, memory usage, and energy consumption, especially when dealing with real-time or portable systems. Due to their relatively small architecture, CNNs are usually preferred in embedded and mobile scenarios. Optimization techniques like pruning, quantization, and knowledge distillation are also used to lessen computational load. These techniques all contribute to models having low-power deployment.

ResNet architectures, on the other hand, are better suited for server-based applications. Due to their availability of resources, it is possible for the execution of more precise models without sacrificing responsiveness. Edge computing platforms such as Google Coral and NVIDIA Jetson are good options for localized processing. Real-time on-

device FER is enabled by these gadgets which lessen dependency on cloud-based systems, enhance user privacy, and reduce latency.

The incorporation of FER into multimodal emotion recognition systems is recognized as another new trend. A more comprehensive understanding of emotional states is provided by these systems through the integration of physiological signals, such as skin conductance or heart rate, with facial analysis and vocal tone.

2.8 Ethical and Bias Considerations

Despite their potential, there are also drawbacks in FER systems. Bias in training data can cause predictions to become skewed, especially when regarding different genders, ages, and ethnic groups. In applications like healthcare and law enforcement, fairness and transparency in model outputs are important to be guaranteed.

The facial regions that impact a model's choice are visualized by interpretability tools such as Grad-CAM and LIME. Regulatory compliance, such as GDPR and AI ethics frameworks, must also guide FER development. The trust of users is gained by requiring openness in the collection, labelling, and use of data.

Cultural sensitivity is considered another important consideration. How facial expressions are interpreted can be greatly affected by cultural differences. Models trained on datasets that are primarily Western may incorrectly classify emotions in non-Western populations. Therefore, it is crucial for FER systems to have different representatives of datasets incorporated.

2.9 Future Directions

Multimodal emotion recognition research is being advanced by fusing visual cues with text, audio, or physiological signals. An additional frontier in which systems are adapted to changing emotional contexts or user feedback for updates is real-time adaptive FER.

Model robustness against occlusions and changes in lighting is also being developed. Advancements in data augmentation, synthetic data generation, and continuous learning is going to be an important factor in shaping the next generation of FER systems.

Emotionally intelligent agents may also be possible through integration with emotional dialogue systems and large language models (LLMs). Context-aware and emotionally responsive interactions can be enabled by such systems, and industries like education, elder care, and customer service can be improved greatly.

2.10 Conclusion

In conclusion, the fundamental and sophisticated methods in FER are represented by CNNs and ResNets, respectively. Greater accuracy and better generalization are provided by ResNets, while effective deployment is best achieved by custom CNNs. More accurate and applicable FER systems are being developed as a result of the combination of attention and temporal modelling as well as hybrid designs.

Ethical considerations should be taken into account, demographic equity should be guaranteed, and interpretability tools should be made use of as the field develops. The careful application of sound foundational design will determine the future impact of FER technologies.

A future in which FER systems will be more accurate, equitable, explicable, and flexible enough to handle a variety of real-world situations is pointed to by deep learning architectures, better dataset accessibility, and ethical design principles.

3.0 Methodology

3.1 Data Selection:

For our training and testing dataset, we have used the FER-2013 dataset which is uploaded on www.kaggle.com. The dataset has many advantages which is the main reason we chose it for our study.

1. Large and rich variety of data: Consists of 7 different main files which represent different emotions. Training dataset contains over 28,000 files and testing dataset contains over 7,000 files.
2. Diverse objects: Each emotion are shown from different angles, and on different people.
3. Standardization of dataset: The dataset is well maintained and contains little to none objects with defects. Additionally, the bounding boxes and segmentation masks are clear, which is ideal for deep learning frameworks.

3.2 Data Preprocessing:

Three main methods are used in data preprocessing: resize, normalization, and data augmentation (rotation/flip).

Resize:

The input images are resized uniformly to 224×224 pixels. This is essential because deep learning models like CNNs and pretrained networks (e.g., ResNet) require fixed input sizes for consistent layer computation. By resizing all images to a standard shape, it ensures compatibility with the convolutional and fully connected layers, and supports efficient batch training.

Data Augmentation (rotation/flip):

Random horizontal flipping and rotation ($\pm 10^\circ$) are applied to the training images to simulate various real-world conditions, such as different facial orientations or slight camera tilts.

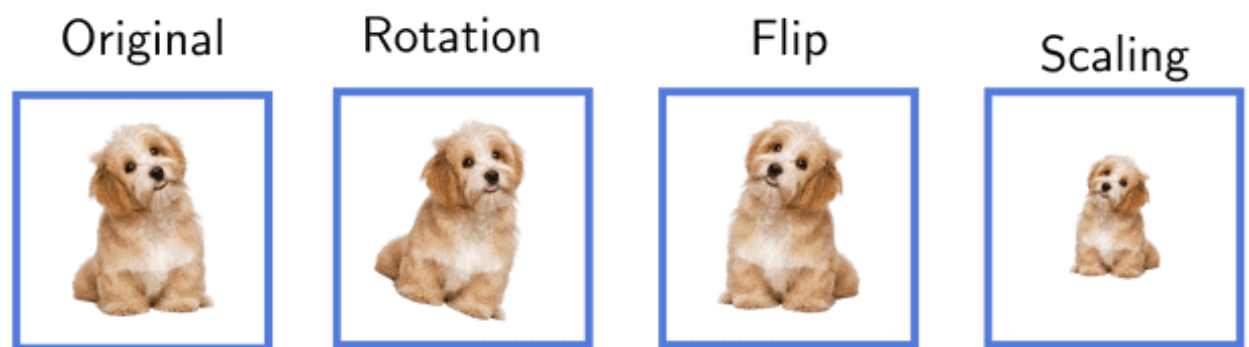
This enhances the robustness of the model by teaching it to generalize across pose variations, thereby reducing overfitting and improving performance on unseen data.

Normalization:

Each image is normalized with a mean of 0.5 and a standard deviation of 0.5 across all RGB channels.

This transforms pixel values from the $[0, 255]$ range to approximately $[-1, 1]$.

Normalization ensures stable gradients, accelerates training convergence, and prevents dominance of certain features due to varying scales.



3.3 Feature Extraction:

EmotionCNN Model

The convolutional feature extraction pipeline begins with input preprocessing. While the input images are grayscale (single channel), they're converted to 3-channel format to maintain compatibility with standard PyTorch transformations and pretrained models. This conversion provides flexibility for potential future modifications involving color processing.

The first convolutional layer (conv1) employs 32 filters of 3×3 kernel size with padding=1. This configuration is particularly significant because:

- The 3×3 kernel size represents the smallest receptive field that can capture center-surround patterns
- Padding=1 ensures spatial dimensions are preserved after convolution

- 32 filters provide sufficient capacity to learn diverse low-level features while remaining computationally efficient

The ReLU activation function (applied during forward pass) introduces crucial nonlinearity, enabling the network to learn complex patterns. Following this, max pooling with 2×2 kernels and stride=2 reduces spatial dimensions by 50%, achieving:

1. Dimensionality reduction for computational efficiency
2. Translation invariance by preserving only the most activated features
3. Progressive expansion of the receptive field

The subsequent convolutional blocks (conv2 and conv3) follow the same pattern but double the number of filters at each stage ($32 \rightarrow 64 \rightarrow 128$). This increasing filter count mirrors the biological visual hierarchy where later processing stages integrate information from larger receptive fields and more complex features.

Hierarchical Feature Learning Mechanism

The network's three convolutional blocks create a powerful feature extraction hierarchy:

1. First Block (conv1):
 - Learns basic edge detectors and gradient filters
 - Operates on the smallest receptive fields (3×3 pixels)
 - Outputs 32 feature maps capturing fundamental texture information
2. Second Block (conv2):
 - Combines low-level features into simple facial components
 - Begins detecting eye corners, mouth curves, and eyebrow shapes
 - 64 filters allow for more specialized feature detectors
3. Third Block (conv3):
 - Synthesizes higher-order facial expression patterns
 - Detects complex configurations like smile arcs or furrowed brows
 - 128 filters enable fine discrimination between similar expressions

The spatial dimensions are as follows:

- Input: 48×48 (after padding)
- After pool1: 24×24
- After pool2: 12×12
- After pool3: 6×6

This progressive down sampling creates an information bottleneck that forces the network to learn the most salient features for emotion discrimination.

3.4 Model Building:

EmotionCNN model

The transition from feature extraction to classification of EmotionCNN occurs at the flattening operation. The 128 feature maps of size 6×6 is reshaped into a single vector of length $128 \times 6 \times 6 = 4608$ dimensions. This high-dimensional representation contains all learned facial expression features in a format suitable for linear classification.

The fully connected layers then perform the actual emotion classification:

1. First Dense Layer (fc1):
 - Projects 4608 dimensions down to 512
 - Applies ReLU activation for nonlinear transformation
 - Serves as an information bottleneck that forces feature compression
2. Dropout Layer:
 - Randomly zeros 50% of activations during training
 - Prevents co-adaptation of features
 - Effectively creates an ensemble of subnetworks
3. Final Dense Layer (fc2):
 - Maps 512 dimensions to 7 output classes

- Produces raw logits (unnormalized scores) for each emotion class
- Relies on CrossEntropyLoss to handle softmax normalization

Consistent use of padding=1 maintains spatial dimensions through convolutional layers, only reducing size via pooling

Computational Complexity

The computational cost can be estimated by examining each layer's operations:

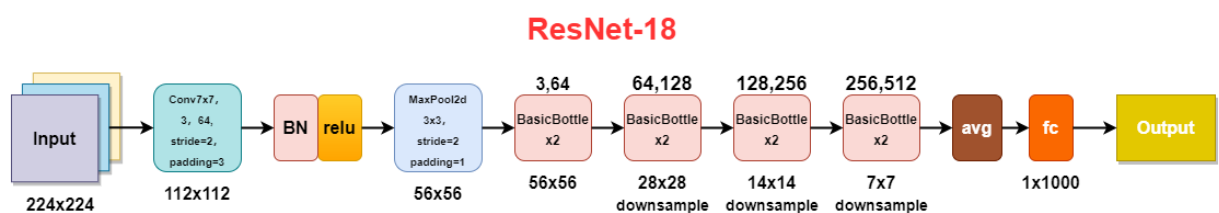
1. conv1: $3 \times 3 \times 3 \times 32 = 864$ parameters
2. conv2: $3 \times 3 \times 32 \times 64 = 18,432$ parameters
3. conv3: $3 \times 3 \times 64 \times 128 = 73,728$ parameters
4. fc1: $4608 \times 512 = 2,359,296$ parameters
5. fc2: $512 \times 7 = 3,584$ parameters

Total parameters: ~2.45 million, with the vast majority (96%) in the first dense layer. This highlights the importance of the convolutional feature extraction in reducing dimensionality before the expensive fully-connected operations.

ResNet-18 model

The ResNet-18 model adapts a proven deep learning architecture for facial emotion analysis. Its modified design includes:

- Input layer processing 48×48 grayscale images (converted to 3-channel)
- Four residual blocks with increasing feature depth
- Custom classification head (7 emotion classes)
- Partial fine-tuning of only the last layers



ResNet-18 model

3.5 Model training:

Loss Function

The loss function measures how well the model performs by comparing its predictions to the true labels. Here, Cross-Entropy Loss is used because the task is classification (predicting emotion classes). It penalizes incorrect predictions and helps guide the model toward better accuracy during training.

Optimization

The Adam optimizer is used to update the model's weights. Adam combines the benefits of momentum and adaptive learning rates, making it efficient for training deep neural networks. The learning rate is set to 0.0001, controlling how much the model adjusts its weights in response to errors.

$$\begin{aligned}
 m_t &= \beta_1 * m_{t-1} + (1 - \beta_1) * g_t \\
 v_t &= \beta_2 * v_{t-1} + (1 - \beta_2) * g_t^2 \\
 \hat{m}_t &= m_t / (1 - \beta_1^t) \\
 \hat{v}_t &= v_t / (1 - \beta_2^t) \\
 \theta &= \theta - (\alpha * \hat{m}_t / \sqrt{(\hat{v}_t + \epsilon)})
 \end{aligned}$$

Adam Optimizer Algorithm

Training Loop

The training loop iterates over the dataset in batches for multiple epochs (in this case, 20). In each iteration:

1. The model processes a batch (train_loader), computes predictions, and calculates loss.

2. Backpropagation (`loss.backward()`) adjusts the model's weights to minimize loss.
3. The optimizer updates the weights (`optimizer.step()`).
4. Training loss is tracked to monitor progress.

Cross-Validation (Validation Step)

Instead of traditional k-fold cross-validation, this code uses a hold-out validation set (separate from training data). After each epoch, the model is evaluated on the `val_loader` to check generalization performance. Validation accuracy is computed by comparing predictions to true labels, helping detect overfitting.

Split Ratio

The dataset is split into training and validation sets in an 80-20 ratio.

3.6 Model Evaluation

The model evaluation checks how well the trained model performs on unseen data. It calculates standard classification metrics and shows where the model makes mistakes.

In our report, we have used accuracy and ROC curve (Receiver Operating Characteristic curve) as a degree of measurement for each emotion class.

Accuracy: This tells us the overall proportion of correctly classified instances by the model across all emotion classes. It indicates how often the model's predictions align with the true emotion labels.

ROC curve (Receiver Operating Characteristic curve): For each emotion class, the ROC curve visualizes the trade-off between the True Positive Rate and the False Positive Rate at various classification thresholds. It helps to understand how well the model can distinguish between positive (the specific emotion class) and negative (all other emotion classes) instances, and its shape provides insight into the model's diagnostic ability across different levels of strictness in classification.

Besides, a confusion matrix is also used to visualize the model's performance as percentages. The diagonal shows correct predictions, while other cells reveal which

emotions get confused. For example, the model might mix up "happy" and "surprise" more often than other pairs.

3.7 Fine Tuning/Hyperparameter tuning

ResNet Model

All layers except layer4 and fc are frozen. This includes the initial conv1 layer, BatchNorm layers and layer1-layer3 residual blocks. This is because these early layers detect generic features (edges, textures) that remain useful for facial expressions without retraining. On the other hand, the last residual block layer4 is unfrozen to adapt high-level features specific to facial expressions. The final classification head is also replaced and trained from scratch (since the original was for 1,000 ImageNet classes).

This selective fine-tuning approach balances efficiency and performance. By freezing ResNet's early layers, we keep their general image processing capabilities intact. Only the last layers are trained, adapting them specifically for emotion recognition while maintaining the model's core visual understanding.

The method helps prevent overfitting on smaller emotion datasets. Since most of the network remains unchanged, we avoid having too many trainable parameters relative to the available training data. The frozen layers continue providing reliable feature extraction while the trainable layers focus on learning emotion-specific patterns.

This approach also saves computation time and resources. Updating only the final layers reduces the training workload compared to retraining the entire network. It delivers good results efficiently by focusing adjustments where they matter most for the emotion recognition task.

4.0 Results

FER dataset:

This dataset consists of series human emotions of facial image like Angry, Disgust, Fear, Happy, Sad, Surprise, and Neutral. We have designed Facial Expression Recognition systems to classify out the human emotions from these facial images.

Data Description:

- **Format:** 48×48-pixel grayscale images
- **Classes:** 7 emotion categories
- **Total Images:** 35,887 examples
- **Split:**
 - Training: 28,709 images (80%)
 - Test: 7,178 images (20%)

For the dataset, we designed CNN model and Resnet model to train and test the model.

Confusion matrix

Axes:

Y-axis (True Label):

The correct emotions of the test facial image.

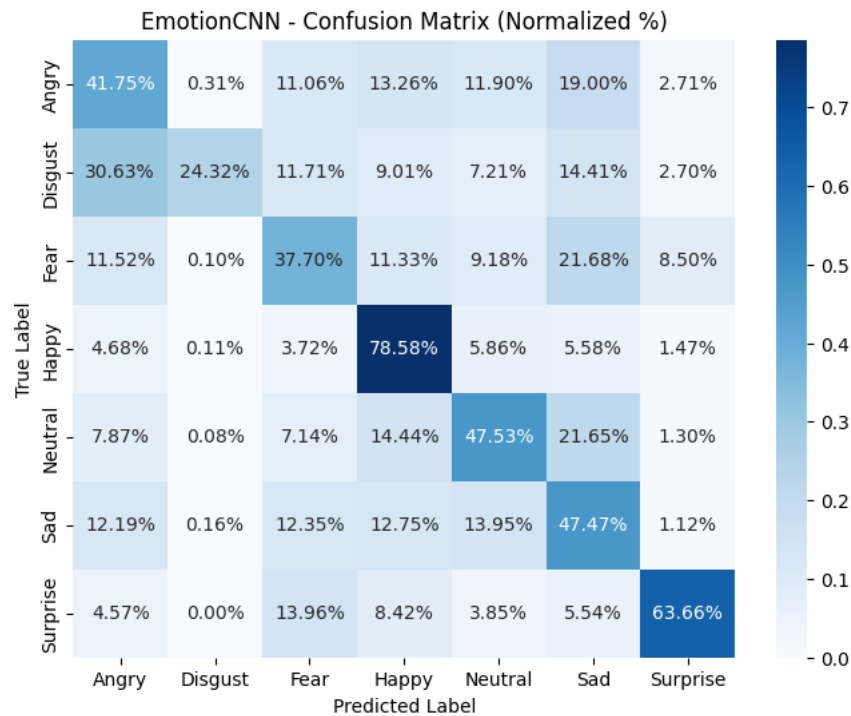
X-axis (Predicted Label):

The predicted facial emotions interpreted by the models.

Aim:

Confusion matrix compared both of the true label and the predicted label to show the performance of the model based on the data inputted.

EmotionCNN model



1. Angry Class

Correct Prediction: 41.75%

Less than half of angry expressions are correctly identified, this may sure show some improvement needed in anger recognition.

Main Misclassifications:

Anger will sometimes be mistaken for with other high-arousal emotions, thus the model may struggle to distinguish between those facial expressions.

2. Disgust Class

Correct Prediction: 24.32%

Very low accuracy, which is the worst-performing category among other emotions.

Main Misclassifications:

Disgust is mainly mistaken for angry; this may be due to the reason that they both have frowning expressions when having this emotion.

3. Fear Class

Correct Prediction: 37.70%

Only one-third of fear expressions are recognized.

Main Misclassifications:

Fear and surprise share similar wide-eyed expressions, causing frequent confusion between these high-arousal states.

4. Happy Class

Correct Prediction: 78.58%

Strong performance with nearly 80% accuracy, making this the second-best recognized emotion.

Main Misclassifications:

The model may confuse with some facial expressions like wrinkled nose or some sad emotions like subtle smile, which is very similar and results in the low accuracy.

5. Neutral Class

Correct Prediction: 47.53%

Moderate performance where about half of neutral expressions are correctly identified.

Main Misclassifications:

The confusion with fear likely occurs in borderline cases with slight eyebrow movements.

6. Sad Class

Correct Prediction: 47.47%

performance with almost half accuracy to identify surprise emotion.

Main Misclassifications:

The bidirectional confusion with happy (an opposite emotion) suggests the model may be over-reliant on mouth features alone.

7. Surprise Class

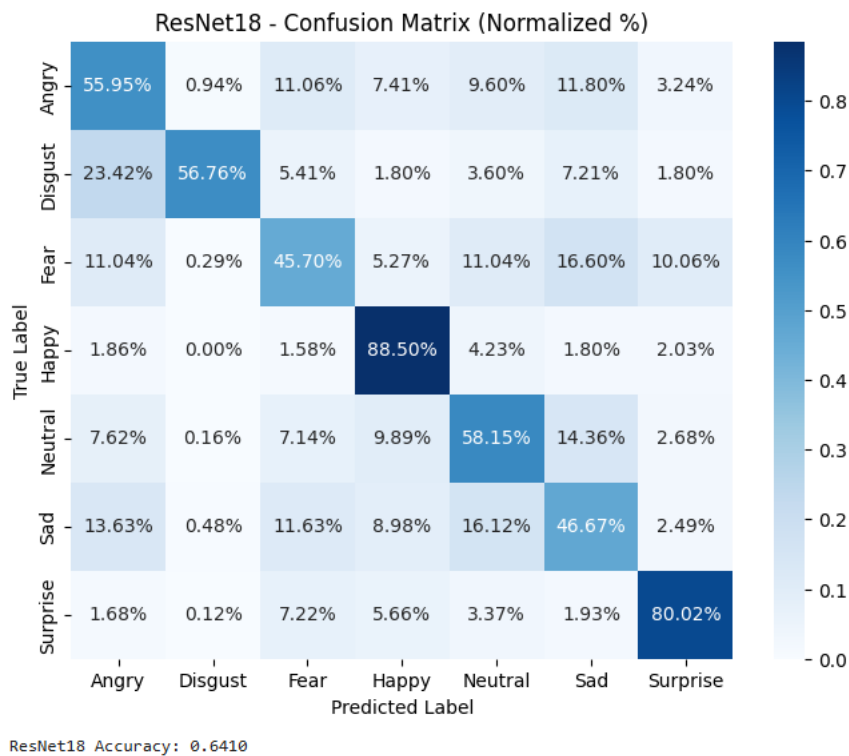
Correct Prediction: 63.66%

Best-performing category with nearly 65% accuracy.

Main Misclassifications:

The unexpected confusion with low-arousal emotions indicates potential issues with eyebrow/forehead feature extraction.

ResNet Model



1. Angry Class

Correct Prediction: 55.95%

Improved from previous model but still misclassifies nearly half of angry expressions.

Main Misclassifications:

Anger continues to be confused with other high-arousal emotions, though less severely than before.

2. Disgust Class

Correct Prediction: 56.76%

Dramatic improvement from previous 24.32%, now the second-best performing class.

Main Misclassifications:

While vastly improved, disgust-anger confusion persists as the primary error case.

3. Fear Class

Correct Prediction: 45.70%

Notable improvement from previous 37.70% accuracy.

Main Misclassifications:

Fear-surprise confusion remains problematic but less severe than in previous model.

4. Happy Class

Correct Prediction: 88.50%

Outstanding performance, now the best-recognized emotion.

Main Misclassifications:

The small confusion with sadness persists but at reduced rates.

5. Neutral Class

Correct Prediction: 58.15%

Significant improvement from previous 47.53% accuracy.

Main Misclassifications:

The neutral confusion persists but at reduced rates where it may be due to the mouth expression

6. Sad Class

Correct Prediction: 47.47%

Improved from previous, but still not exceeding 50% accuracy.

Main Misclassifications:

The model now confuses surprise more with sadness than other emotions.

7. Surprise Class

Correct Prediction: 80.02%

Improved from previous accuracy from around 65 % to 80%, maintaining strong performance.

Main Misclassifications:

Sad-surprise confusion emerges as a new pattern in this model.

ROC Curve

The **ROC curve (Receiver Operating Characteristic curve)** is a graphical representation of a binary classifier's performance across all classification thresholds. There are two plots on the axis.

Y-axis :

True Positive Rate (TPR)

X-axis :

False Positive Rate (FPR)

Definitions:

True Positive Rate (TPR):

It can be also called as **Sensitivity** or **Recall**.

Formula:

TPR=True Positives (TP)+False Negatives (FN)

Aim:

TPR measures the amounts of actual positives which were correctly interpreted by the model, which indicate the accuracy of the model.

False Positive Rate (FPR):

Formula:

FPR=False Positives (FP)+True Negatives (TN)

Aim:

FPR measures the amounts of actual negatives which were wrongly interpreted by the model as positive.

ROC Curves mechanic:

Each point on the curve corresponds to a specific **classification threshold** (e.g., 0.5 for binary decisions).

Threshold changes in TPR and FPR:

The higher the threshold, the fewer positives predicted → **Low TPR, High FPR** (lower-right corner).

The lower threshold , the more positives predicted → **High TPR, Low FPR** (upper-left corner).

Interpreting the ROC Curve:

Top-left corner (0,1):

The Ideal point which **TPR =1 and FPR = 0**, which indicate a **perfect classifier**

Diagonal line (y=x):

Represents a **random guess classifier** (e.g., AUC = 0.5).

Curve above the diagonal:

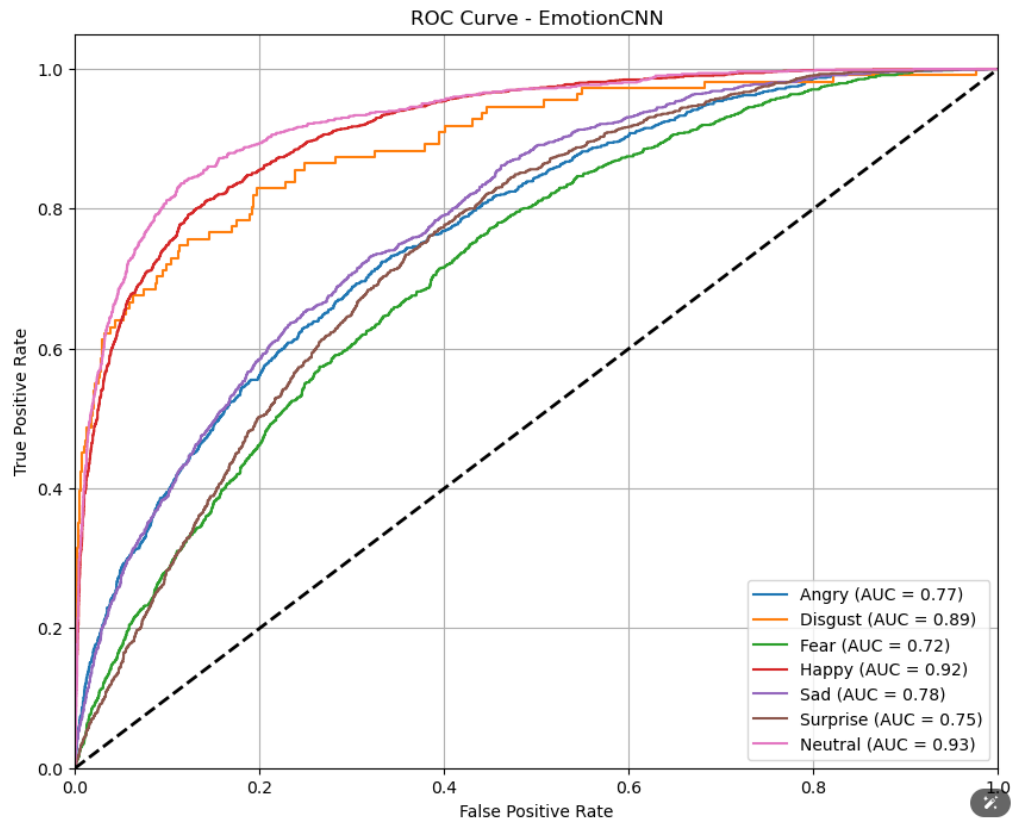
Indicates better-than-random performance.

Area Under the Curve (AUC):

Quantifies the overall performance of a model whether it operates well or not.

(AUC=1: perfect; AUC=0.5: random).

EmotionCNN Model



AUC

Emotion	AUC Score	Interpretation
Angry	0.77	Good
Disgust	0.89	Excellent
Fear	0.72	Fair
Happy	0.92	Excellent
Sad	0.78	Good
Surprise	0.75	Fair to Good
Neutral	0.93	Excellent

Interpretation :

Strongest Classes (High AUC):

Neutral (0.93) and Happy (0.92): These are the highest AUC among other emotions which means the model can easily and highly accurately predict these emotions from the training set, with an almost perfect classifier score, above 0.90.

Disgust (0.89): Despite that Disgust has a slightly lower AUC compared to neutral and happy, the AUC value is still considered relatively high and thus this model is still very good to predict the emotion correctly.

Weaker Classes (Lower AUC):

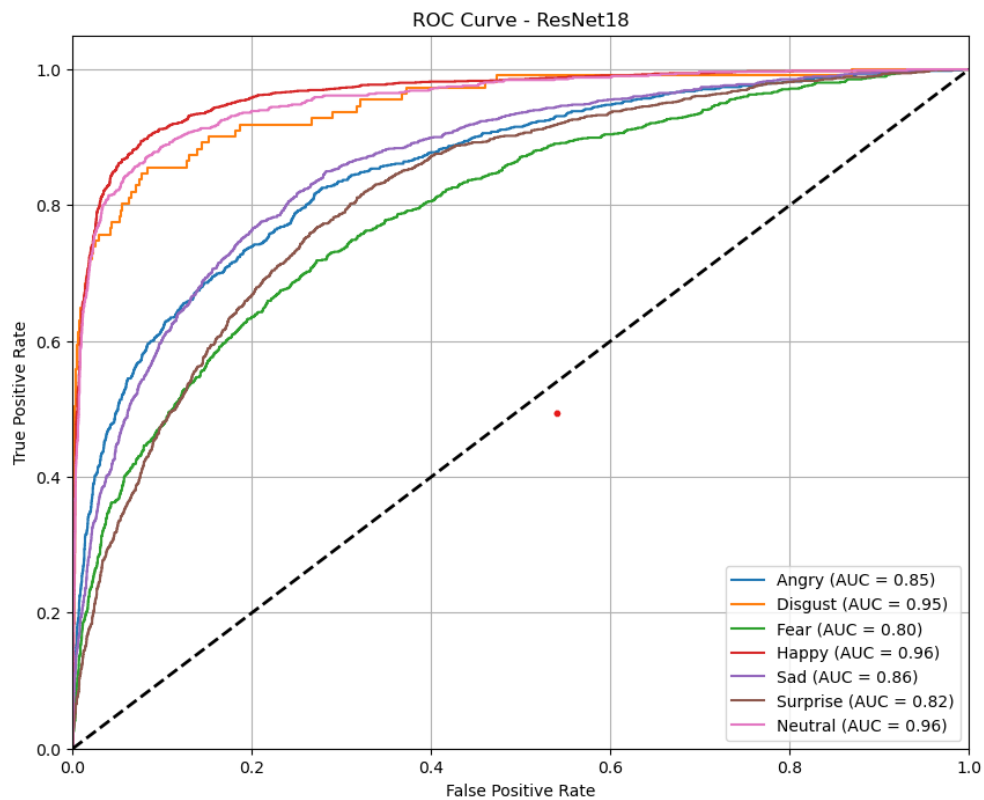
Angry (0.77): This emotion has a lower AUC, may be caused by some of different facial expression which it has similar characteristics with other emotions .

Fear (0.72) and **Surprise (0.75)**: Both of them have a lowest AUC compared to others . These may because both of the emotions are confused with each other visually to the similar classes like wide eyes and also tense expression).

Overall:

The CNN model is still considered good as all of the AUC is above 0.7, which implied that the model still holds as a satisfying performance model.

ResNet Model



AUC

Emotion	AUC Score	Interpretation
Angry	0.85	Good
Disgust	0.95	Excellent
Fear	0.80	Fair to Good

Emotion	AUC Score	Interpretation
Happy	0.96	Perfect
Sad	0.86	Good
Surprise	0.82	Fair to Good
Neutral	0.96	Perfect

Interpretation :

Strongest Classes (High AUC):

Neutral (0.96) and Happy (0.96): These are the highest AUC among other emotions which means that the model is very excellent to be used to predict these emotions by considering its performance. Furthermore, the characteristics of both emotions are clearly distinct from other emotions, which thus easily predicted by the model.

Disgust (0.95): Despite that Disgust has a slightly lower AUC compared to neutral and happy, the AUC value is still considered relatively excellent as it is above 0.90 threshold which means disgust is also very easily interpreted by this model.

Weaker Classes (Lower AUC):

Angry (0.85): This emotion has a lower AUC emotion compared to neutral, happy and disgust but this model still considered performed well as it still hit around 0.85.

Fear (0.80) and Surprise (0.82): Both of them have a lowest AUC compared to others . This is may still because of the same problem where both of them has a lot of similar class with other expression as the characteristics of these emotions may differs a lot, and so harder to be interpreted by the model, so the model performs a lower accuracy on these emotions.

Overall:

The CNN model is still considered excellent as all of the AUC is above 0.8, which implied that the model is an excellent performance model to be used as an emotion recognition tool.

Model testing on real-world images

Predicting: angry Bong 2.jpg

Predicted Emotion: Angry



The model has successfully predicted one of our members emotions which is angry. This means that the model although doesn't have the highest accuracy in emotion but it's still capable to predict angry emotions in reality.

5.0 Conclusion, limitations, and future work

Using the FER-2013 dataset as a benchmark, this project demonstrated the design, implementation, and comparative assessment of two deep learning models for facial expression recognition (FER): EmotionCNN and ResNet-18. Faster inference and lower computational costs were two benefits that made EmotionCNN appropriate for resource-constrained environments. ResNet-18, on the other hand, continuously produced better accuracy and robustness, especially when it came to recognizing complex emotional states. Effective adaptation to the unique requirements of the FER domain was made possible by the pretrained ResNet model's performance being greatly improved through the use of transfer learning and selective fine-tuning.

Despite these developments, a number of restrictions were found. The main causes of the frequent misclassification of emotions like "disgust," "fear," and "angry" were identified as the imbalanced distribution of emotion classes in the dataset and overlapping facial features. Moreover, problems with generalization are still noted. Under real-world circumstances, such as occlusions, inconsistent lighting, or motion artifacts, consistent functioning of the model might not be achieved when expressions from non-Western populations are applied. The value of expanding testing in more naturalistic settings and diversifying datasets is emphasized by these elements.

Training datasets should be grown in future research to include a wider variety of emotional expressions that are more evenly distributed. Studying hybrid architectures, like CNN-LSTM or 3D CNNs, has the potential to reveal temporal patterns that static models might miss. Classification performance could be further improved by integrating attention mechanisms and multimodal inputs, such as physiological signals or vocal tones. Ongoing efforts to address interpretability and fairness will be equally important. These components are essential for the ethical implementation of FER systems, especially in fields where decisions impacted by these technologies have serious repercussions, such as healthcare, education, and security.

6.0 Member's contributions

Name and Student ID	Tasks (Code)	Tasks (Report)
Bong Xuan Jing	Coding and training the EmotionCNN model, providing and testing on own images, coding real-time image classification	Proofreading report
Sean Wong Wee Kai	Providing and testing on own images	Writing introduction, methodology and dataset sections
Woon Kai Fan	Importing and training the ResNet model, writing code for model evaluation	Writing the appendix
Alikhan	Writing code for data pre-processing	Writing literature review, conclusion, limitations and future work section
Onn Heng Hwa	Writing code for data pre-processing	Writing literature review, conclusion, limitations and future work section
Lim Zee Ern	Writing code for model evaluation	Writing results section

7.0 References

Ayeni, J. A. (2022). Convolutional Neural Network (CNN): The architecture and applications. *Applied Journal of Physical Science*, 4(4), 42–50.
<https://www.researchgate.net/publication/373715986>

Bi, K., Liao, J., Gao, D., & Yin, R. (2025). *A comparative study on facial expression recognition using MobileNetV2, VGG-16, ResNet and Swin Transformer*. SPIE International Conference on Affective Computing and Intelligent Interaction.
<https://doi.org/10.1117/12.3060400>
(Conference version available at SPIE Digital Library: mdpi-res.com+2spiedigitallibrary.org+2researchgate.net+2spiedigitallibrary.org)

Deramgozin, M. M., Yaghoobi, M., & Ghassemi, A. (2023). Lightweight attention-based CNN on embedded systems for emotion recognition. *ICECS 2023*, 167–171.
<https://www.semanticscholar.org/paper/dc495bb9474796d7e161fa9207237824dfb681e>

Dhoot, A., Patel, K., & Nair, R. (2023). 2D CNN vs. 3D CNN: An empirical study. *MoSICom* 2023.
<https://www.semanticscholar.org/paper/43e18b09a69672ac36bbe56800794178d5dd7557>

Gao, Z. (2024). Comparison of CNN and ResNet neural networks on facial expression recognition. *Highlights in Science, Engineering and Technology*, 18(2), 88–101.
<https://www.semanticscholar.org/paper/16c38224ce4d93a1e5d69c660cf8bffb92de22bc>

Gursesli, M. C., Tunc, E., & Ozkan, H. (2024). Facial emotion recognition (FER) through custom lightweight CNN model. *IEEE Access*, 12, 45,543–45,559.
<https://www.semanticscholar.org/paper/30051e453320f351b2113f6f0d4702a91a77eb21>

Gupta, S., Roy, A., & Basu, S. (2022). Facial emotion recognition in real-time learning environments. *Multimedia Tools and Applications*, 81(4), 5671–5690.

<https://www.semanticscholar.org/paper/5db429d455835fb25b285156c29e3ce09a1285cd>

Ibrahim, S. P. S., Kamil, D. M., & Rahman, H. A. (2024). Comparative analysis of VGG-16, ResNet-50, and a custom CNN architecture. *ICDICI* 2024. <https://www.semanticscholar.org/paper/544cc3fcc2dc4752bf8bbe551296084522413eff>

Mehta, S., & Bhalla, A. (2024). Advanced deep learning for emotion recognition. *ICMACC* 2024. <https://www.semanticscholar.org/paper/24fbc76d742a75205a512cc89c1b3a3301b5f839>

Qian, C., Luo, Y., & He, Z. (2024). Analysis of deep learning algorithms for emotion classification. *Big Data and IoT Conference*. <https://www.semanticscholar.org/paper/8d1c3629cab575bf34d570485ad174586bcb2ccf>

Reddy, K. A., Naidu, V. P., & Thomas, S. (2023). Performance enhancement in facial emotion classification through noise-injected FERCNN model. *Ingénierie des systèmes d'information*, 36, 202–217. <https://www.semanticscholar.org/paper/b782a901be479d5518d53c55dba058ff4fee1b79>

8.0 Dataset and code links

Code:

[Final Project 4.ipynb](#)

Dataset:

FER-2013 (Kaggle image dataset)

facial recognition (image dataset from group members)

9.0 Important Code and Appendix

construct the dataset class

```
class FERDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df.reset_index(drop=True)
        self.transform = transform

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        pixels = np.fromstring(self.df.loc[idx, 'pixels'], sep=' ', dtype=np.uint8)
        image = pixels.reshape(48, 48) # grayscale
        image = Image.fromarray(image).convert("RGB") # convert to RGB
        label = int(self.df.loc[idx, 'emotion'])

        if self.transform:
            image = self.transform(image)

        return image, label
```

Image preprocessing which adjusts the image into a more readable form to the model

```
# Image preprocessing: training set (including data augmentation)
train_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
])

# Test set preprocessing: no enhancement
test_transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
])
```

Load the Fer2013dataset and split into train and test dataset

```

from torchvision import datasets, transforms
from torch.utils.data import DataLoader, Subset
from sklearn.model_selection import train_test_split

# Paths
train_dir = r"C:\Users\user\Downloads\archive\train"
test_dir = r"C:\Users\user\Downloads\archive\test"

# Load datasets
train_dataset = datasets.ImageFolder(train_dir, transform=train_transform)
test_dataset = datasets.ImageFolder(test_dir, transform=test_transform)

# Split training dataset into training and validation sets
train_idx, val_idx = train_test_split(
    list(range(len(train_dataset))),
    test_size=0.2,
    stratify=train_dataset.targets, # ensures balanced split across classes
    random_state=42
)

train_split = Subset(train_dataset, train_idx)
val_split = Subset(train_dataset, val_idx)

# Define DataLoaders
train_loader = DataLoader(train_split, batch_size=64, shuffle=True)
val_loader = DataLoader(val_split, batch_size=64, shuffle=False)
test_loader = DataLoader(test_dataset, batch_size=64, shuffle=False)

```

Model Building for the CNN model

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class EmotionCNN(nn.Module):
    def __init__(self):
        super(EmotionCNN, self).__init__()

        # Convolutional Layers
        self.conv1 = nn.Conv2d(in_channels=3, out_channels=32, kernel_size=3, padding=1)
        self.pool1 = nn.MaxPool2d(kernel_size=2, stride=2)

        self.conv2 = nn.Conv2d(32, 64, 3, padding=1)
        self.pool2 = nn.MaxPool2d(2, 2)

        self.conv3 = nn.Conv2d(64, 128, 3, padding=1)
        self.pool3 = nn.MaxPool2d(2, 2)

        # Flatten + Fully connected Layers
        self.fc1 = nn.Linear(128 * 28 * 28, 512)
        self.dropout = nn.Dropout(0.5)
        self.fc2 = nn.Linear(512, 7) # Output: 7 emotion classes

    def forward(self, x):
        # Convolution + ReLU + Pooling
        x = self.pool1(F.relu(self.conv1(x)))
        x = self.pool2(F.relu(self.conv2(x)))
        x = self.pool3(F.relu(self.conv3(x)))

        # Flatten
        x = x.view(x.size(0), -1)

        # Fully connected Layers
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x) # No softmax here; use CrossEntropyLoss Later

        return x

```

Model training for the CNN model

```

from tqdm.notebook import tqdm

def train_model(model, train_loader, val_loader, optimizer, loss_fn, device, num_epochs=30, save_path="best_model.pth"):
    train_losses, val accuracies = [], []
    best_acc = 0.0

    for epoch in range(num_epochs):
        model.train()
        running_loss = 0.0

        pbar = tqdm(train_loader, desc=f"Epoch {epoch+1}/{num_epochs}")
        for images, labels in pbar:
            images, labels = images.to(device), labels.to(device)
            optimizer.zero_grad()
            outputs = model(images)
            loss = loss_fn(outputs, labels)
            loss.backward()
            optimizer.step()
            running_loss += loss.item()

        avg_train_loss = running_loss / len(train_loader)
        train_losses.append(avg_train_loss)

        model.eval()
        correct, total = 0, 0
        with torch.no_grad():
            for images, labels in val_loader:
                images, labels = images.to(device), labels.to(device)
                outputs = model(images)
                _, predicted = torch.max(outputs, 1)
                total += labels.size(0)
                correct += (predicted == labels).sum().item()
        val_acc = correct / total
        val accuracies.append(val_acc)

        print(f"\nEpoch {epoch+1}: Train Loss = {avg_train_loss:.4f}, Val Acc = {val_acc:.4f}")

        if val_acc > best_acc:
            best_acc = val_acc
            torch.save(model.state_dict(), save_path)
            print(f"Saved best model to {save_path}")

    return train_losses, val accuracies

```

The model is trained 20 times and it will save the highest accuracy for each training


```
#start training cnn model
```

```
train_losses_cnn, val_acc_cnn = train_model(
    emotioncnn, train_loader, val_loader,
    optimizer_cnn, criterion, device,
    num_epochs=20, save_path="best_cnn_model.pth")
```

```
Epoch 1/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 1: Train Loss = 1.7084, Val Acc = 0.3835
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 2/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 2: Train Loss = 1.5683, Val Acc = 0.4289
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 3/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 3: Train Loss = 1.4857, Val Acc = 0.4549
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 4/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 4: Train Loss = 1.4279, Val Acc = 0.4605
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 5/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 5: Train Loss = 1.3833, Val Acc = 0.4777
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 6/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 6: Train Loss = 1.3438, Val Acc = 0.4854
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 7/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 7: Train Loss = 1.3100, Val Acc = 0.5031
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 8/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 8: Train Loss = 1.2716, Val Acc = 0.4956
```

```
Epoch 9/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 9: Train Loss = 1.2443, Val Acc = 0.5106
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 10/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 10: Train Loss = 1.2129, Val Acc = 0.5185
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 11/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 11: Train Loss = 1.1836, Val Acc = 0.5211
```

```
Saved best model to best_cnn_model.pth
```

```
Epoch 12/20: 0%|          | 0/359 [00:00<?, ?it/s]
```

```
Epoch 13: Train Loss = 1.1223, Val Acc = 0.5273
Saved best model to best_cnn_model.pth
Epoch 14/20: 0%|          | 0/359 [00:00<?, ?it/s]

Epoch 14: Train Loss = 1.0971, Val Acc = 0.5310
Saved best model to best_cnn_model.pth
Epoch 15/20: 0%|          | 0/359 [00:00<?, ?it/s]

Epoch 15: Train Loss = 1.0602, Val Acc = 0.5246
Epoch 16/20: 0%|          | 0/359 [00:00<?, ?it/s]

Epoch 16: Train Loss = 1.0273, Val Acc = 0.5357
Saved best model to best_cnn_model.pth
Epoch 17/20: 0%|          | 0/359 [00:00<?, ?it/s]

Epoch 17: Train Loss = 1.0018, Val Acc = 0.5374
Saved best model to best_cnn_model.pth
Epoch 18/20: 0%|          | 0/359 [00:00<?, ?it/s]

Epoch 18: Train Loss = 0.9681, Val Acc = 0.5388
Saved best model to best_cnn_model.pth
Epoch 19/20: 0%|          | 0/359 [00:00<?, ?it/s]

Epoch 19: Train Loss = 0.9307, Val Acc = 0.5361
Epoch 20/20: 0%|          | 0/359 [00:00<?, ?it/s]

Epoch 20: Train Loss = 0.9073, Val Acc = 0.5376
```

Model building for ResNet

```
from torchvision import models
import torch.nn as nn

resnet = models.resnet18(pretrained=True)

for name, param in resnet.named_parameters():
    if "layer4" in name or "fc" in name:
        param.requires_grad = True
    else:
        param.requires_grad = False

resnet.fc = nn.Linear(resnet.fc.in_features, 7)
resnet = resnet.to(device)
```

Model training for ResNet

```
import torch.optim as optim

criterion = nn.CrossEntropyLoss()
optimizer_resnet = optim.Adam(filter(lambda p: p.requires_grad, resnet.parameters()), lr=0.0001, weight_decay=1e-5)

from torchvision import transforms
transform_train = transforms.Compose([
    transforms.Resize((48, 48)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(10),
    transforms.ToTensor(),
    transforms.Normalize((0.5, ), (0.5, ))
])

transform_test = transforms.Compose([
    transforms.Resize((48, 48)),
    transforms.ToTensor(),
    transforms.Normalize((0.5, ), (0.5, ))
])

train_losses_resnet, val_acc_resnet = train_model(
    resnet,
    train_loader,
    val_loader,
    optimizer_resnet,
    criterion,
    device,
    num_epochs=20,
    save_path="best_resnet_model_v2.pth"
)
```

We trained the model for 20 times as Resnet will be more accurate to predict if the training time increases, and the model will also save the highest accuracy after each training.

Epoch 1: Train Loss = 1.2853, Val Acc = 0.5731 Saved best model to best_resnet_model_v2.pth	Epoch 11: Train Loss = 0.3867, Val Acc = 0.6223
Epoch 2: Train Loss = 1.0440, Val Acc = 0.5994 Saved best model to best_resnet_model_v2.pth	Epoch 12: Train Loss = 0.3450, Val Acc = 0.6411 Saved best model to best_resnet_model_v2.pth
Epoch 3: Train Loss = 0.9392, Val Acc = 0.6106 Saved best model to best_resnet_model_v2.pth	Epoch 13: Train Loss = 0.3110, Val Acc = 0.6290
Epoch 4: Train Loss = 0.8564, Val Acc = 0.6217 Saved best model to best_resnet_model_v2.pth	Epoch 14: Train Loss = 0.2796, Val Acc = 0.6231
Epoch 5: Train Loss = 0.7725, Val Acc = 0.6001	Epoch 15: Train Loss = 0.2577, Val Acc = 0.6278
Epoch 6: Train Loss = 0.7009, Val Acc = 0.6224 Saved best model to best_resnet_model_v2.pth	Epoch 16: Train Loss = 0.2377, Val Acc = 0.6343
Epoch 7: Train Loss = 0.6276, Val Acc = 0.6113	Epoch 17: Train Loss = 0.2166, Val Acc = 0.6364
Epoch 8: Train Loss = 0.5511, Val Acc = 0.6348 Saved best model to best_resnet_model_v2.pth	Epoch 18: Train Loss = 0.1883, Val Acc = 0.6461 Saved best model to best_resnet_model_v2.pth
Epoch 9: Train Loss = 0.4918, Val Acc = 0.6341	Epoch 19: Train Loss = 0.1847, Val Acc = 0.6297
Epoch 10: Train Loss = 0.4313, Val Acc = 0.6367 Saved best model to best_resnet_model_v2.pth	Epoch 20: Train Loss = 0.1671, Val Acc = 0.6355

Model evaluation and visualization of the dataset

Confusion matrix

```

from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np

def evaluate_model(model, test_loader, class_names, device, model_name="Model"):
    model.eval()
    y_true, y_pred = [], []

    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)
            y_true.extend(labels.cpu().numpy())
            y_pred.extend(predicted.cpu().numpy())

    #classification reports
    print(f"\n{model_name} - Classification Report:")
    print(classification_report(y_true, y_pred, target_names=class_names))

    # Confusion Matrix
    cm = confusion_matrix(y_true, y_pred)
    plt.figure(figsize=(8, 6))
    sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
                xticklabels=class_names, yticklabels=class_names)
    plt.title(f"{model_name} - Confusion Matrix")
    plt.xlabel("Predicted Label")
    plt.ylabel("True Label")
    plt.show()

    # Accuracy score
    accuracy = np.mean(np.array(y_true) == np.array(y_pred))
    print(f"{model_name} Accuracy: {accuracy:.4f}")
    return accuracy

```

Summary report for CNN and ResNet

CNN

```
#CNN best model
emotioncnn.load_state_dict(torch.load("best_cnn_model.pth"))
emotioncnn = emotioncnn.to(device)
acc_cnn = evaluate_model(emotioncnn, test_loader, class_names, device, model_name="EmotionCNN")
```

```
EmotionCNN - Classification Report:
      precision    recall  f1-score   support

     Angry         0.43      0.42      0.42         958
    Disgust         0.81      0.23      0.36         111
       Fear         0.41      0.34      0.37        1024
      Happy         0.72      0.77      0.74        1774
        Sad         0.47      0.53      0.50        1233
    Surprise         0.43      0.39      0.41        1247
     Neutral         0.66      0.72      0.69         831

 accuracy                   0.54         7178
  macro avg                 0.56         7178
 weighted avg              0.54         7178
```

ResNet

EmotionCNN Accuracy: 0.5410

```
[18]: #ResNet18 best model
resnet.load_state_dict(torch.load("best_resnet_model_v2.pth"))
resnet = resnet.to(device)
acc_resnet = evaluate_model(resnet, test_loader, class_names, device, model_name="ResNet18")
```

```
ResNet18 - Classification Report:
      precision    recall  f1-score   support

     Angry         0.55      0.57      0.56         958
    Disgust         0.68      0.57      0.62         111
       Fear         0.55      0.45      0.49        1024
      Happy         0.87      0.83      0.85        1774
        Sad         0.55      0.67      0.60        1233
    Surprise         0.52      0.50      0.51        1247
     Neutral         0.75      0.78      0.77         831

 accuracy                   0.65         7178
  macro avg                 0.64         7178
 weighted avg              0.65         7178
```

ROC Curve for further visualization on the accuracy of the model

ROC Curve

```
#ROC Curve
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize
from sklearn.metrics import RocCurveDisplay

def plot_roc_curve_multiclass(model, test_loader, class_names, device, model_name="Model"):
    model.eval()
    y_score = []
    y_true = []

    with torch.no_grad():
        for images, labels in test_loader:
            images = images.to(device)
            outputs = model(images)
            y_score.extend(outputs.cpu().numpy())
            y_true.extend(labels.numpy())

    y_score = np.array(y_score)
    y_true_bin = label_binarize(y_true, classes=list(range(len(class_names))))
    fpr = dict()
    tpr = dict()
    roc_auc = dict()
    n_classes = len(class_names)

    for i in range(n_classes):
        fpr[i], tpr[i], _ = roc_curve(y_true_bin[:, i], y_score[:, i])
        roc_auc[i] = auc(fpr[i], tpr[i])

    # Plot all ROC curves
    plt.figure(figsize=(10, 8))
    for i in range(n_classes):
        plt.plot(fpr[i], tpr[i], label=f"{class_names[i]} (AUC = {roc_auc[i]:.2f})")

    plt.plot([0, 1], [0, 1], "k--", lw=2)
    plt.xlim([0.0, 1.0])
    plt.ylim([0.0, 1.05])
    plt.xlabel("False Positive Rate")
    plt.ylabel("True Positive Rate")
    plt.title(f"ROC Curve - {model_name}")
    plt.legend(loc="lower right")
    plt.grid()
    plt.show()
```

Final result to see the comparison of the two model

```
print("\nFinal Comparison Summary:")
print(f"EmotionCNN Accuracy: {acc_cnn:.4f}")
print(f"ResNet18 Accuracy : {acc_resnet:.4f}")

if acc_resnet > acc_cnn:
    print("ResNet18 performed better overall.")
else:
    print("EmotionCNN performed better overall.")
```

```
Final Comparison Summary:
EmotionCNN Accuracy: 0.5410
ResNet18 Accuracy : 0.6470
ResNet18 performed better overall.
```

APPENDIX 1: MARKING RUBRICS

Component Title	Coding and Report			Percentage (%)	70%	
Criteria	Score and Descriptors				Weight (%)	Marks
	Excellent (8-10)	Good-Average (4-8)	Need Improvement (0-4)			
Introduction	Clear and precise introduction of the topic, methods, and scope. The introduction gives an impressively clear idea of the work done in the project.	The students introduce the context, and it is fairly clear.	The introduction is not clear as it does not provide enough information to understand what the work of the project is about.		10	
Literature review	The literature review is comprehensive and written critically. It demonstrates an understanding of the related studies and methods.	The literature review is appropriate, and it clearly reviews a few related studies and methods.	The literature review is unorganized with limited information.		10	
Methodology /model and dataset	The dataset collection and preparation process are clearly explained with some dataset analysis. The method is well described with justifications. Some innovative ideas are put into it. In addition, the selection of evaluation measurements is good.	The dataset and method are described appropriately.	The description of the used dataset and method is not clear. It is difficult to understand the method or the dataset preparation process.		10	
Results	The results are illustrated and described clearly, and the figures and tables are appropriately formatted. Some comparisons are made and described.	Training and testing results are provided and somewhat described well.	Some results are provided, but it is not easy to interpret them.		10	
Conclusion, limitations, and future work	A clear and critical conclusion of the project is found. The students are able to point out the findings, limitations, and future work.	The conclusion is somewhat written well. The limitations and future work are given.	The conclusion needs more effort. The limitations or future work are lacking or not included.		10	
Coding	The coding is precise and excellent. All the ideas described in the methodology section are implemented correctly.	The code works and it has a good style.	The code is unorganized and contains unnecessary parts. Also, some codes are not working properly.		10	
Report quality	The students show excellence in writing a clear and precise report. The lecturer's instructions are followed precisely. A comprehensive list of references is found, which are cited in-text and formatted correctly.	The report is reasonably organized and understandable. References are sufficient, but with minor inconsistencies.	The report is unclear/unorganized. The references are not formatted correctly.		10	
TOTAL						
Component Title	Presentation			Percentage (%)	30%	
Criteria	Score and Descriptors				Weight (%)	Marks
	Excellent (8-10)	Good-Average (4-8)	Need Improvement (0-4)			
Presentation quality	The presenters show remarkable skill in demonstrating. The quality is excellent, and the students are able to cover a proper amount of content within the given time limit.	The presenters' sound is clear and understandable. The duration is less than fifteen minutes (in the case of three members).	The quality of the presentation needs improvement. No proper presentation slides are used.		10	
The contents flow and teamwork	The flow of the contents is very easy to follow and well-thought-out. From the presentation, you can see that there is a lot of teamwork effort has been put into preparing the presentation. Also, the students can state/demonstrate their contribution to the project.	The flow of the contents is good and can be followed. The contents/time of the presentation is somehow divided appropriately among the group members.	The flow of the contents needs improvement. Teamwork is lacking.		10	
Presentation contents	The students manage to present all the most important contents of their work such as the dataset, model, results, and findings, which can precisely demonstrate the hard work done to complete the project.	The contents are appropriate, and the students can demonstrate their understanding of these contents.	The contents of the presentation need improvement, and the students show a lack in demonstrating their understanding of what is being presented. Some technical terms and descriptions are incorrect.		10	
TOTAL						

Note to students: Please print out and attach this appendix together with the submission of coursework